

Figure 1: How the shell works

Csh and Tcsh: Developers have written large parts of the Linux operating system in the C and C++ languages. Using the C syntax as a model, Bill Joy at Berkeley University developed the *C-shell* (*csh*) in 1978. Ken Greer, working at Carnegie-Mellon University, took *csh* concepts a step forward with a new shell called *tcsh*, which Linux systems now offer. *Tcsh* fixed problems in *csh* and added command completion, in which the shell makes educated guesses as you type, based on your system's directory structure and files. *Tcsh* does not run bash scripts, as the two have substantial differences.

Ksh: David Korn developed the *Korn shell*, or *ksh*, about the time *tcsh* was introduced. *Ksh* is compatible with *sh* and *bash*. *Ksh* improves on the Bourne shell by adding floating-point arithmetic, job control, command aliasing and command completion. AT&T held proprietary rights to *ksh* until 2000, when it became open source.

Zsh: The Z shell is the ultimate for feature-hungry users. If you can think of something that a shell ought to do, the Z shell probably does it. All that power comes at a price. For example, the *zsh* executable is nearly four times larger than the *ksh* executable and almost three times larger than the *sh* executable (Bourne shell). *Zsh* was developed by Paul Falstad.

If you want to know which shell is installed in your computer, use the following command:

```
$ ls /bin/ | grep sh
```

You can try all the shells available in your computer by just typing the name of the shell in the terminal.



Note: There are many other shells out there such as *fish* (the friendly interactive shell), *POSIX*, *Ash*, etc. I have given a brief on the ones that everyone knows a little about.

Why bash?

By default, we have the *bash* shell for users and the Bourne shell (*sh*) for the root user in Ubuntu, which has been the default since version 6.10. Fedora (Red Hat) also uses *bash* as the default shell. The major reason to switch the default shell is the efficiency of the *bash*. One day, every distro will have to shift to *zsh* as it is more efficient than *bash*.

The main features that have made *bash* the default shell in most distributions are:

- *bash* implements command aliases, and the alias and unalias built-ins

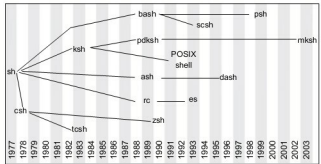


Figure 2: Different shells

- *bash* includes a built-in help for quick reference to shell facilities
- *bash* can be customised easily by using the `~/bashrc` file
- *bash* has process substitution
- *bash* has indirect variable expansion using `${!word}`
- *bash* implements *csh*-like history expansion

You can get a list of all the major differences from the following link:

http://www.gnu.org/software/bash/manual/html_node/Major-Differences-From-The-Bourne-Shell.html

Shell scripting

These are the rules that must not be forgotten while writing a script:

- The first line should be `#!/bin/sh`
- You must change the permissions of the file to execute it — `chmod a+x filename`
- Always use comments to explain the code; if the first character is '#', then the line is treated as a comment.
- Use the `echo` command to explain what is happening in the background when the shell is running.
- The extension of the file need not be `.sh`

Here's my first shell code:

```
#!/bin/sh
# my first script

echo "Hello World"
```

The first line says that this is a code that needs to be executed using the Bourne shell. The second line is a comment as it starts with #. *Echo* just prints whatever exists within the " ".

```
#!/bin/sh
# greeting script

echo "Hi $USER"
echo "Have a nice day !!!!!!!!!!"
```

Echo substitutes the `$USER` with the username logged in and prints the greetings.